基于信用卡逾期数据的Spark数据处理与分析

https://dblab.xmu.edu.cn/blog/2738/

本案例以和鲸社区的信用卡评分模型构建数据为数据集，以Python为编程语言，使用大数据框架Spark对数据进行处理分析，并对分析结果进行可视化。

一、实验环境

（1）Linux：Ubuntu 18.04
（2）Hadoop3.1.3 （查看安装教程）
（3）Python：3.6
（4）Spark：2.4.0 （查看安装教程）
（5）Jupyter Notebook （查看安装和使用方法教程）

安装上述环境后，为了支持Python可视化分析，还需要执行如下命令安装pyecharts组件：

```
pip3 install pyecharts==1.7.0
```

Shell 命令

二、数据集

1. 数据集下载

本次实验数据集来自和鲸社区的信用卡评分模型构建数据，以数据集cs-training.csv为分析主体，其中共有15万条记录，11列属性。

每个数据包含以下字段：

字段名称 字段含义 例子

（1）SeriousDlqin2yrs 是否逾期 0,1

（2）RevolvingUtilizationOfUnsecuredLines 信用卡和个人信贷额度的总余额 0.766126609
（3）Age 年龄 45,20,30
（4）NumberOfTime30-59DaysPastDueNotWorse 借款人逾期30-59天的次数 0,2,3
（5）DebtRatio 负债比率 0.802982129
（6）MonthlyIncome 月收入 9120,3000
（7）NumberOfOpenCreditLinesAndLoans 未偿还贷款数量 ,0,4,13
（8）NumberOfTimes90DaysLate 借款人逾期90天以上的次数 0,1,3
（9）NumberRealEstateLoansOrLines 房地产贷款的数量 3,6
（10）NumberOfTime60-89DaysPastDueNotWorse 借款人逾期60-89天的次数 0,3
（11）NumberOfDependents 家庭中的家属人数 0,1,3

2. 数据预处理

本次实验采用pandas库对数据进行预处理。在实验中，不对信用卡和个人信贷额度的总余额、负债比率、未偿还贷款数量、逾期90天以上的次数这4个属性进行处理分析。
具体处理步骤如下：
（1）读取数据
（2）查看数据是否具有重复值，去除重复值
（3）查看各字段缺失率，缺失值以均值填充
（4）选取要研究的属性，删除不研究的属性
（5）保存文件到本地
使用代码文件data_preprocessing.py对数据预处理，运行data_preprocessing.py文件的步骤如下：

```
python ~/OverDue/data_preprocessing.py
```
Shell 命令

代码文件data_preprocessing.py的内容如下：

```
import pandas as pd
```

```python
# 读取数据
df = pd.read_csv("~/Desktop/cs-training.csv")

# 去除重复值
df.duplicated()
df.drop_duplicates()

# 查看各字段缺失率
df.info()
# 缺失值按均值填充
for col in list(df.columns[df.isnull().sum() > 0]):
    mean_val = df[col].mean()
    df[col].fillna(mean_val, inplace=True)

# 删除不分析的列
columns = ["RevolvingUtilizationOfUnsecuredLines","DebtRatio","NumberOfOpenCreditLinesAndLoans","NumberOfTimes90DaysLate"]
df.drop(columns,axis=1,inplace=True)

# 保存到本地
df.to_csv("~/OverDue/data.csv")
```

Python

3. 将文件上传至HDFS文件系统

将本地文件系统的数据集“data.csv”上传到HDFS文件系统中，路径为“/OverDue/data.csv”。具体命令如下：

```shell
# 启动Hadoop
cd /usr/local/hadoop
./sbin/start-dfs.sh
# 在HDFS文件系统中创建/OverDue目录
./bin/hdfs dfs -mkdir /data
# 上传文件到HDFS文件系统中
```

```
./bin/hdfs dfs -put ~/OverDue/data.csv /
OverDue/data.csv
```

Shell 命令

三、使用Spark对数据处理分析

我们将采用Python编程语言和Spark大数据框架对数据集“data.csv”进行处理分析，具体步骤如下：

（1）读取HDFS文件系统中的数据文件，生成DataFrame
（2）修改列名
（3）本次信用卡逾期的总体统计
（4）年龄与本次信用卡逾期的结合统计
（5）两次逾期记录与本次信用卡逾期的结合统计
（6）房产抵押数量与本次信用卡逾期的结合统计
（7）家属人数与本次信用卡逾期的结合统计
（8）月收入与本次信用卡逾期的结合统计
（9）将统计数据返回给数据可视化文件data_web.py

代码文件data_analysis.py的内容如下：

```
from pyspark.sql import SparkSession
from pyspark import SparkContext,SparkConf
from pyspark.sql import Row
from pyspark.sql.types import *
from pyspark.sql import functions

def analyse(filename):
    # 读取数据
    spark = SparkSession.builder.config(conf =
SparkConf()).getOrCreate()
     df =
spark.read.format("csv").option("header","true
").load(filename)

     # 修改列名
     df =
df.withColumnRenamed('SeriousDlqin2yrs','y')
```

```
    df =
df.withColumnRenamed('NumberOfTime30-59DaysPas
tDueNotWorse','30-59days')
    df =
df.withColumnRenamed('NumberOfTime60-89DaysPas
tDueNotWorse','60-89days')
    df =
df.withColumnRenamed('NumberRealEstateLoansOrL
ines','RealEstateLoans')
    df =
df.withColumnRenamed('NumberOfDependents','fam
ilies')

    # 返回data_web.py的数据列表
    all_list = []
    # 本次信用卡逾期分析
    # 共有逾期10026人，139974没有逾期，总人数
150000
    total_y = []
    for i in range(2):
        total_y.append(df.filter(df['y'] ==
i).count())
    all_list.append(total_y)

    # 年龄分析
    df_age = df.select(df['age'],df['y'])
    agenum = []
    bin = [0,30,45,60,75,100]
    # 统计各个年龄段的人口
    for i in range(5):

agenum.append(df_age.filter(df['age'].between(
bin[i],bin[i+1])).count())
    all_list.append(agenum)
    # 统计各个年龄段逾期与不逾期的数量
    age_y = []
    for i in range(5):
```

```
        y0 =
df_age.filter(df['age'].between(bin[i],bin[i+1
])).\
            filter(df['y']=='0').count()
        y1 =
df_age.filter(df['age'].between(bin[i],bin[i+1
])).\
            filter(df['y']=='1').count()
        age_y.append([y0,y1])
    all_list.append(age_y)

    # 有逾期记录的人的本次信用卡逾期数量
    df_pastDue =
df.select(df['30-59days'],df['60-89days'],df['
y'])
    # 30-59有23982人，4985逾期，18997不逾期
    numofpastdue = []

numofpastdue.append(df_pastDue.filter(df_pastD
ue['30-59days'] > 0).count())
    y_numofpast1 = []
    for i in range(2):
        x =
df_pastDue.filter(df_pastDue['30-59days'] >
0).\
            filter(df_pastDue['y'] ==
i).count()
        y_numofpast1.append(x)
    # 60-89有7604人，2770逾期，4834不逾期

numofpastdue.append(df_pastDue.filter(df_pastD
ue['60-89days'] > 0).count())
    y_numofpast2 = []
    for i in range(2):
        x =
df_pastDue.filter(df_pastDue['60-89days'] >
0).\
```

```
            filter(df_pastDue['y'] ==
i).count()
        y_numofpast2.append(x)
    # 两个记录都有的人有4393人，逾期1907，不逾期
2486

numofpastdue.append(df_pastDue.filter(df_pastD
ue['30-59days'] > 0).

filter(df_pastDue['60-89days'] > 0).count())
    y_numofpast3 = []
    for i in range(2):
        x =
df_pastDue.filter(df_pastDue['30-59days'] >
0).\
            filter(df_pastDue['60-89days'] >
0).filter(df_pastDue['y'] == i).count()
        y_numofpast3.append(x)
    all_list.append(numofpastdue)
    all_list.append(y_numofpast1)
    all_list.append(y_numofpast2)
    all_list.append(y_numofpast3)

    # 房产抵押数量分析
    df_Loans =
df.select(df['RealEstateLoans'],df['y'])
    # 有无抵押房产人数情况
    numofrealandnoreal = []

numofrealandnoreal.append(df_Loans.filter(df_L
oans['RealEstateLoans']==0).count())

numofrealandnoreal.append(df_Loans.filter(df_L
oans['RealEstateLoans']>0).count())
    all_list.append(numofrealandnoreal)
    ## 房产无抵押共有56188人，逾期4672人，没逾期
51516人
```

```
    norealnum = []
    for i in range(2):
        x =
df_Loans.filter(df_Loans['RealEstateLoans']==0
).\
            filter(df_Loans['y'] ==
i).count()
        norealnum.append(x)
    all_list.append(norealnum)
    # 房产抵押共有93812人，逾期5354人，不逾期88458
人
    realnum = []
    for i in range(2):
        x =
df_Loans.filter(df_Loans['RealEstateLoans']>0)
.\
            filter(df_Loans['y'] ==
i).count()
        realnum.append(x)
    all_list.append(realnum)

    # 家属人数分析
    df_families =
df.select(df['families'],df['y'])
     # 有无家属人数统计
     nofamiliesAndfamilies = []

nofamiliesAndfamilies.append(df_families.filte
r(df_families['families']>0).count())

nofamiliesAndfamilies.append(df_families.filte
r(df_families['families']==0).count())
     all_list.append(nofamiliesAndfamilies)
     # 有家属59174人，逾期4752人，没逾期54422人
     y_families = []
```

```
y_families.append(df_families.filter(df_families['families']>0).

filter(df_families['y']==0).count())

y_families.append(df_families.filter(df_families['families']>0).

filter(df_families['y']==1).count())
    all_list.append(y_families)
    # 没家属90826人，逾期5274人，没逾期85552人
    y_nofamilies = []

y_nofamilies.append(df_families.filter(df_families['families']==0).

filter(df_families['y']==0).count())

y_nofamilies.append(df_families.filter(df_families['families']==0).

filter(df_families['y']==1).count())
    all_list.append(y_nofamilies)

    # 月收入分析
    df_income = df.select(df['MonthlyIncome'],df['y'])
    # 获取平均值，其中先返回Row对象，再获取其中均值
    mean_income = df_income.agg(functions.avg(df_income['MonthlyIncome'])).head()[0]
    # 收入分布，105854人没超过均值6670，44146人超过均值6670
    numofMeanincome = []
```

```
numofMeanincome.append(df_income.filter(df['MonthlyIncome'] < mean_income).count())

numofMeanincome.append(df_income.filter(df['MonthlyIncome'] > mean_income).count())
    all_list.append(numofMeanincome)
    # 未超过均值的逾期情况分析,97977人没逾期，7877人逾期
    y_NoMeanIncome = []

y_NoMeanIncome.append(df_income.filter(df['MonthlyIncome'] < mean_income).filter(df['y']==0).count())

y_NoMeanIncome.append(df_income.filter(df['MonthlyIncome'] < mean_income).filter(df['y']==1).count())
    all_list.append(y_NoMeanIncome)
    # 超过均值的逾期情况分析，41997人没逾期，2149人逾期
    y_MeanIncome = []

y_MeanIncome.append(df_income.filter(df['MonthlyIncome'] > mean_income).filter(df['y']==0).count())

y_MeanIncome.append(df_income.filter(df['MonthlyIncome'] > mean_income).filter(df['y']==1).count())
    all_list.append(y_MeanIncome)

    # 数据可视化data_web.py
    return all_list
```

Python

四、数据可视化

1. 可视化工具与代码

选择使用python第三方库pyecharts作为可视化工具，其中pyecharts版本为1.7.0。采用其中的柱状图和饼图来详细展现分析结果。
代码文件data_web.py的内容如下：

```
from pyecharts.charts import Bar
from pyecharts.charts import Pie
from pyecharts.charts import Page
from pyecharts import options as opts
import data_analysis
# ---------总体逾期人数情况---------------
def draw_total(total_list):
    attr = ["未逾期人数", "逾期人数"]
    pie = (
        Pie()
            .add("总体逾期人数", [list(z) for z
in zip(attr,total_list)])
            .set_global_opts(title_opts=opts.
TitleOpts(title="总体逾期人数分布"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
    )
    return pie
# ---------年龄与逾期人数情况---------------
def draw_age(age_list,y_ageList):
    total_pie = draw_total(all_list[0])
    attr = ["0-30", "30-45", "45-60",
"60-75", "75-100"]
    y0_agenum = []
```

```
    y1_agenum = []
    for i in range(5):
        y0_agenum.append(y_ageList[i][0])
        y1_agenum.append(y_ageList[i][1])

    bar = (
        Bar()
            .add_xaxis(attr)
            .add_yaxis("人数分布", age_list)
            .add_yaxis("未逾期人数分布",
y0_agenum)
            .add_yaxis("逾期人数分布",
y1_agenum)
            .set_global_opts(title_opts=opts.
TitleOpts(title="各年龄段逾期情况"))
    )
    attr = ["未逾期","逾期"]
    pie1 = (
        Pie()
            .add("0-30年龄段", [list(z) for z
in zip(attr,y_ageList[0])])
            .set_global_opts(title_opts=opts.
TitleOpts(title="0-30年龄段逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
    )
    pie2 = (
        Pie()
            .add("30-45年龄段", [list(z) for z
in zip(attr,y_ageList[1])])
```

```
            .set_global_opts(title_opts=opts.
TitleOpts(title="30-45年龄段逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
    )
    pie3 = (
        Pie()
            .add("45-60年龄段", [list(z) for z
in zip(attr,y_ageList[2])])
            .set_global_opts(title_opts=opts.
TitleOpts(title="45-60年龄段逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
    )
    pie4 = (
        Pie()
            .add("60-75年龄段", [list(z) for z
in zip(attr,y_ageList[3])])
            .set_global_opts(title_opts=opts.
TitleOpts(title="60-75年龄段逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),
```

```
label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
        )
        pie5 = (
            Pie()
                .add("75-100年龄段", [list(z) for
z in zip(attr,y_ageList[4])])
                .set_global_opts(title_opts=opts.
TitleOpts(title="75-100年龄段逾期情况"))
                .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
        )

        page = Page()
        page.add(bar)
        page.add(total_pie)
        page.add(pie1)
        page.add(pie2)
        page.add(pie3)
        page.add(pie4)
        page.add(pie5)
        page.render('age_OverDue.html')

# --------逾期记录与逾期人数情况--------------
def
draw_pastdue(numofpastdue,pastdue1num,pastdue2
num,pastdue12num):
    total_pie = draw_total(all_list[0])
```

```
    attr = ["有30-59days逾期记录的人数", "有60-89days逾期记录的人数", "有长短期逾期记录的人数"]
    bar = (
        Bar()
            .add_xaxis(attr)
            .add_yaxis("人数", numofpastdue)
            .set_global_opts(title_opts=opts.TitleOpts(title="有逾期记录的人数"))
    )
    attr = ["未逾期","逾期"]
    pie1 = (
        Pie()
            .add("有短期逾期记录的人的逾期情况", [list(z) for z in zip(attr,pastdue1num)])
            .set_global_opts(title_opts=opts.TitleOpts(title="有短期逾期记录的人的逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item", formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c} ({d}%)")
        )
    )
    pie2 = (
        Pie()
            .add("有长期逾期记录的人的逾期情况", [list(z) for z in zip(attr,pastdue2num)])
            .set_global_opts(title_opts=opts.TitleOpts(title="有长期逾期记录的人的逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item", formatter="{a} <br/>{b}: {c} ({d}%)"),
```

```
label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
        )
        pie3 = (
            Pie()
                .add("长短期逾期记录都有的人的逾期情
况", [list(z) for z in
zip(attr,pastdue12num)])
                .set_global_opts(title_opts=opts
.TitleOpts(title="长短期逾期记录都有的人的逾期情
况"))
                .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
        )
        page = Page()
        page.add(bar)
        page.add(total_pie)
        page.add(pie1)
        page.add(pie2)
        page.add(pie3)
        page.render('pastDue_OverDue.html')
    # ---------房产抵押与逾期人数情况---------------
    def
draw_realestateLoans(numofrealornoreal,y_norea
lnum,y_realnum):
        total_pie = draw_total(all_list[0])
        attr = ["无房产抵押人数", "有房产抵押人数"]
        bar = (
            Bar()
```

```
            .add_xaxis(attr)
            .add_yaxis("人数",
numofrealornoreal)
            .set_global_opts(title_opts=opts
.TitleOpts(title="房产抵押人数分布"))
        )
        attr = ["未逾期","逾期"]
        pie1 = (
            Pie()
            .add("无房产抵押的人的逾期情况",
[list(z) for z in zip(attr,y_norealnum)])
            .set_global_opts(title_opts=opts
.TitleOpts(title="无房产抵押的人的逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
        )
        pie2 = (
            Pie()
            .add("有房产抵押的人的逾期情况",
[list(z) for z in zip(attr,y_realnum)])
            .set_global_opts(title_opts=opts
.TitleOpts(title="有房产抵押的人的逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
        )
```

```
    page = Page()
    page.add(bar)
    page.add(total_pie)
    page.add(pie1)
    page.add(pie2)

page.render('realestateLoans_OverDue.html')

# ---------家属人数与逾期人数情况----------------
def
draw_families(nofamiliesAndfamilies,y_families
,y_nofamilies):
    total_pie = draw_total(all_list[0])
    attr = ["有家属人数", "无家属人数"]
    bar = (
        Bar()
            .add_xaxis(attr)
            .add_yaxis("人数",
nofamiliesAndfamilies)
            .set_global_opts(title_opts=opts
.TitleOpts(title="有无家属人数分布"))
    )
    attr = ["未逾期","逾期"]
    pie1 = (
        Pie()
            .add("无家属的人的逾期情况",
[list(z) for z in zip(attr,y_nofamilies)])
            .set_global_opts(title_opts=opts
.TitleOpts(title="无家属的人的逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
        )
```

```
    )
    pie2 = (
        Pie()
            .add("有家属的人的逾期情况",
[list(z) for z in zip(attr,y_families)])
            .set_global_opts(title_opts=opts
.TitleOpts(title="有家属的人的逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
    )
    page = Page()
    page.add(bar)
    page.add(total_pie)
    page.add(pie1)
    page.add(pie2)
    page.render('families_OverDue.html')
# --------月收入与逾期人数情况--------------
def
draw_income(numofMeanincome,y_NoMeanIncome,y_M
eanIncome):
    total_pie = draw_total(all_list[0])
    attr = ["未超过均值收入人数", "超过均值收入人
数"]
    bar = (
        Bar()
            .add_xaxis(attr)
            .add_yaxis("人数",
numofMeanincome)
            .set_global_opts(title_opts=opts
.TitleOpts(title="有无超过均值收入人数分布"))
    )
```

```
    attr = ["未逾期","逾期"]
    pie1 = (
        Pie()
            .add("未超过均值收入的人的逾期情况",
[list(z) for z in zip(attr,y_NoMeanIncome)])
            .set_global_opts(title_opts=opts
.TitleOpts(title="未超过均值收入的人的逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
    )
    pie2 = (
        Pie()
            .add("超过均值收入的人的逾期情况",
[list(z) for z in zip(attr,y_MeanIncome)])
            .set_global_opts(title_opts=opts
.TitleOpts(title="超过均值收入的人的逾期情况"))
            .set_series_opts(

tooltip_opts=opts.TooltipOpts(trigger="item",
formatter="{a} <br/>{b}: {c} ({d}%)"),

label_opts=opts.LabelOpts(formatter="{b}: {c}
({d}%)")
            )
    )
    page = Page()
    page.add(bar)
    page.add(total_pie)
    page.add(pie1)
    page.add(pie2)
    page.render('meanIncome_OverDue.html')
```

```
if __name__ == '__main__':
    print("开始总程序")
    Filename = "/OverDue/data.csv"
    all_list = 
data_analysis.analyse(Filename)
    # 年龄与是否逾期情况
    draw_age(all_list[1],all_list[2])
    # 有无逾期记录与是否逾期情况

draw_pastdue(all_list[3],all_list[4],all_list[
5],all_list[6])
    # 房产抵押数量与是否逾期情况

draw_realestateLoans(all_list[7],all_list[8],a
ll_list[9])
    # 家属人数与是否逾期情况

draw_families(all_list[10],all_list[11],all_li
st[12])
    # 月收入与是否逾期情况

draw_income(all_list[13],all_list[14],all_list
[15])
    print("结束总程序")
```

Python

2. 数据可视化结果

我们从年龄、两次逾期记录、房产抵押数量、家属人数、月收入5个方面来分析统计，得到5个直观的结果统计界面。
将代码文件data_web.py提交到spark-submit后，会在OverDue目录下得到5个.html文件，点击浏览就会得到分析结果。
运行步骤如下：

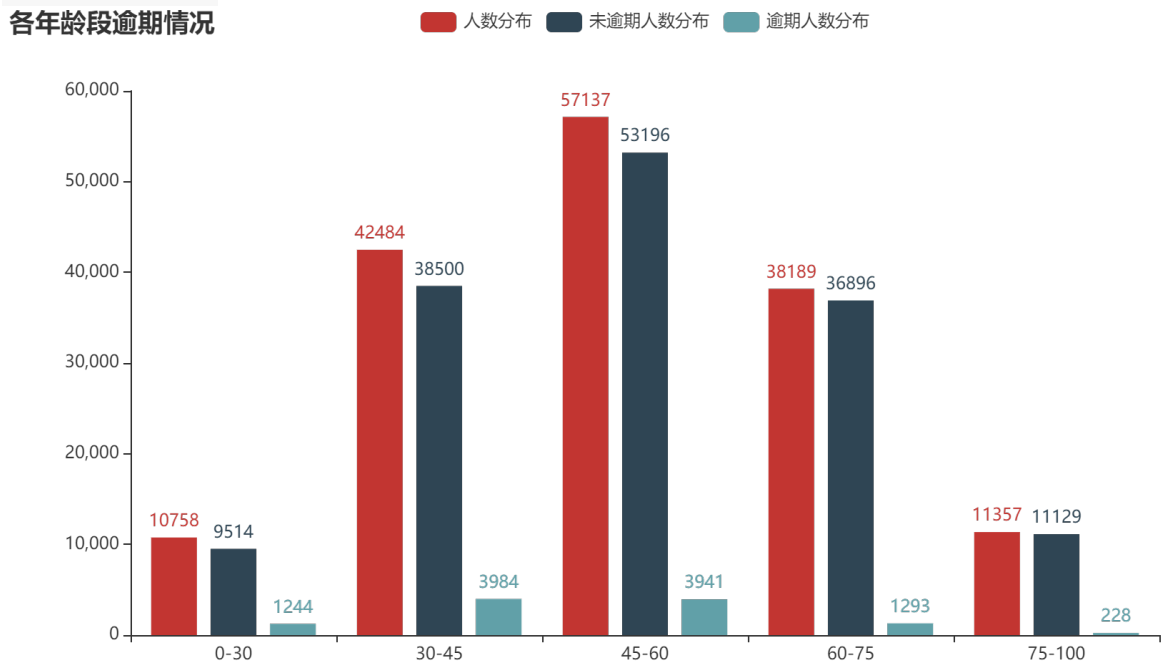
```
# 进入OverDue目录
cd ~/OverDue
```

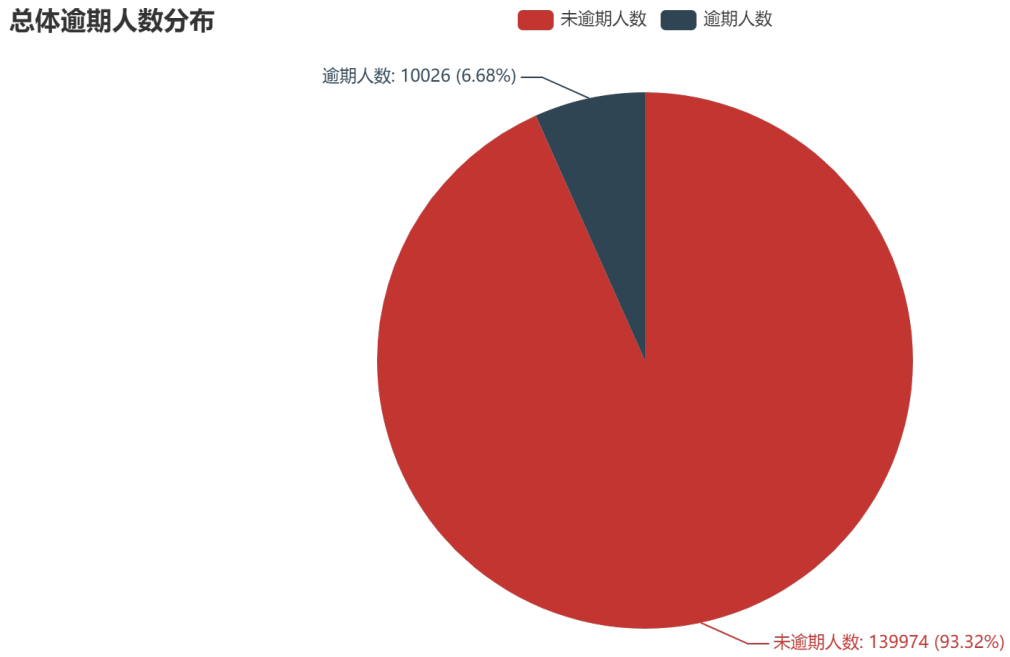
```
# 提交data_web.py文件到spark-submit
/usr/local/spark/bin/spark-submit --master
local ~/OverDue/data_web.py
```

Shell 命令

可视化结果如下：

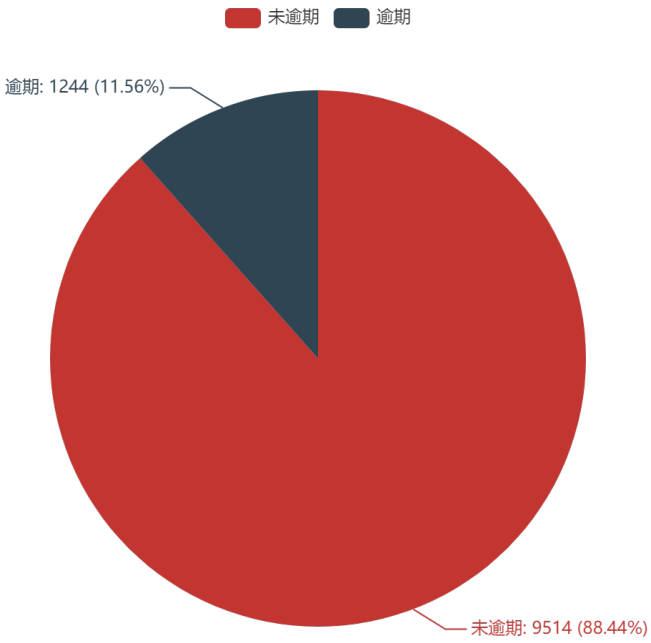
2.1 总体



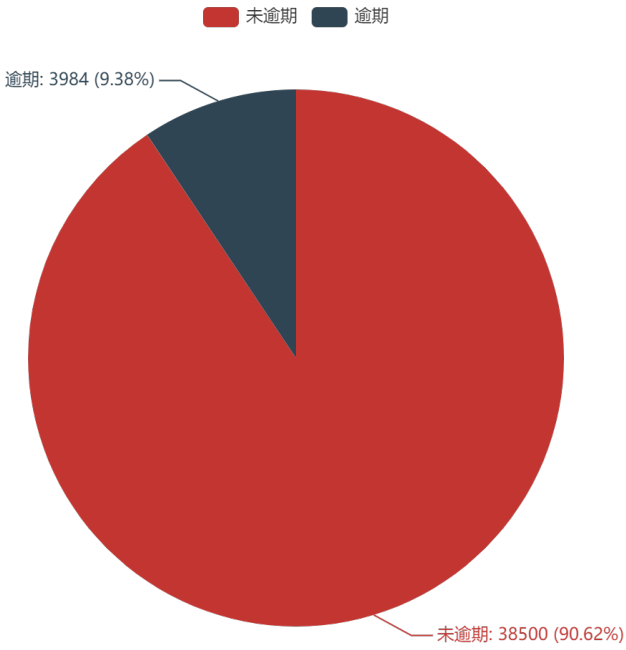


2.2 年龄

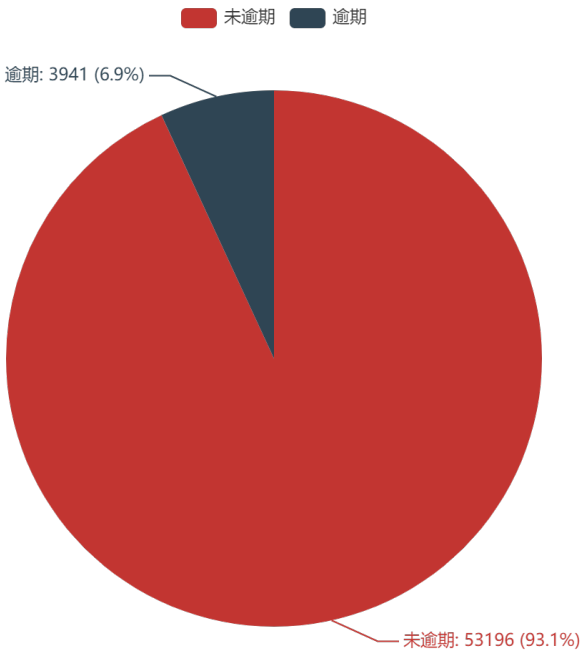
0-30年龄段逾期情况



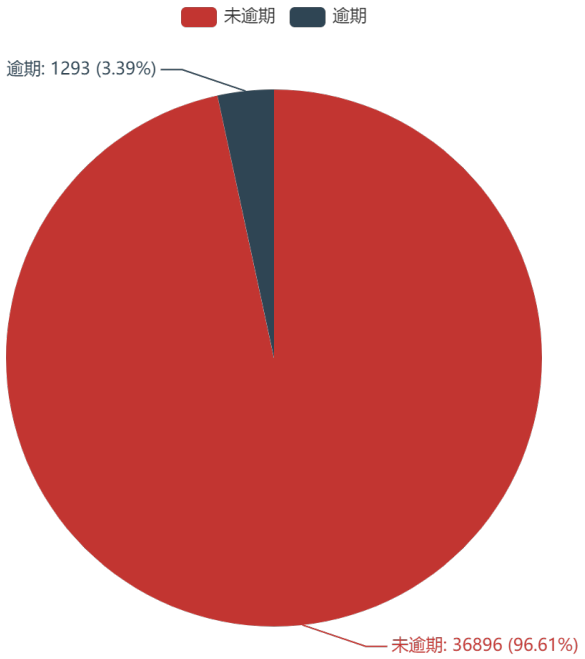
30-45年龄段逾期情况

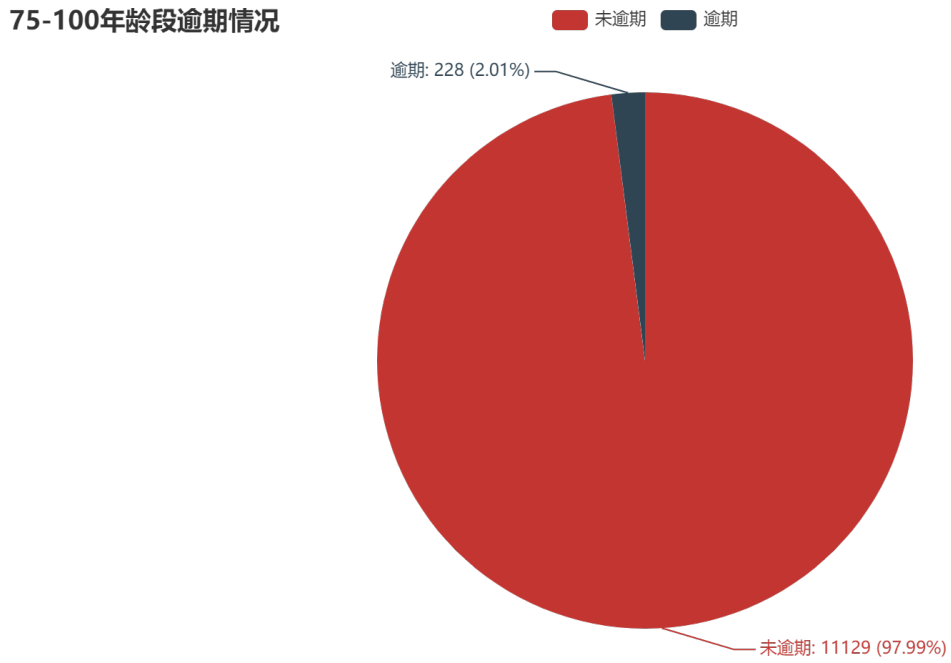


45-60年龄段逾期情况



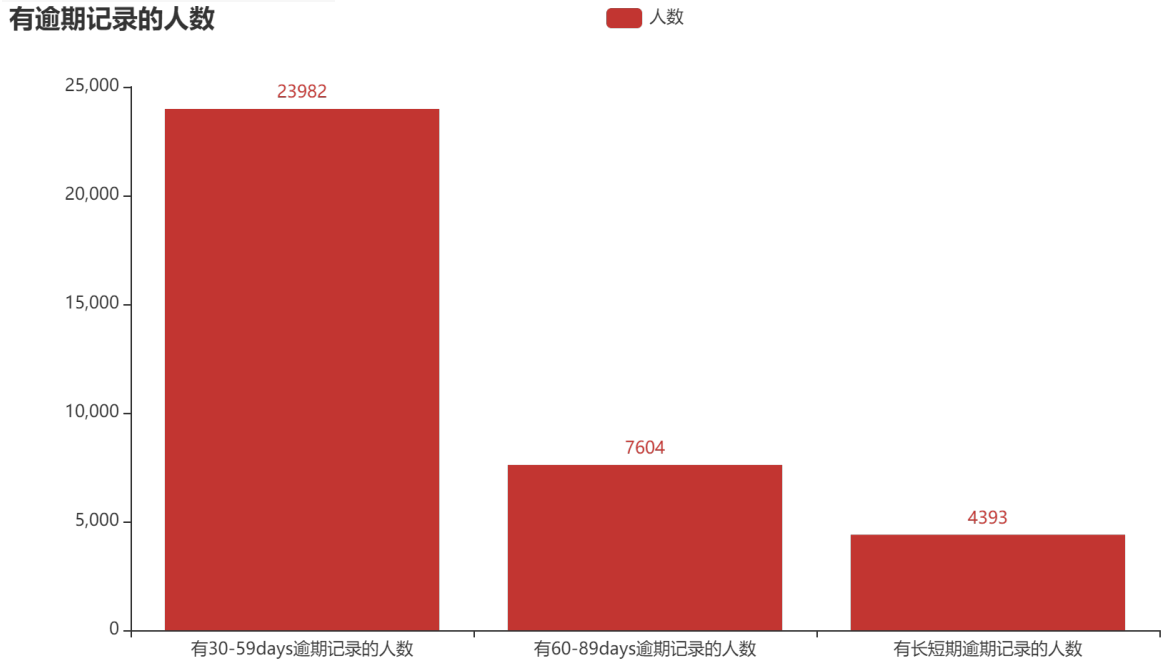
60-75年龄段逾期情况



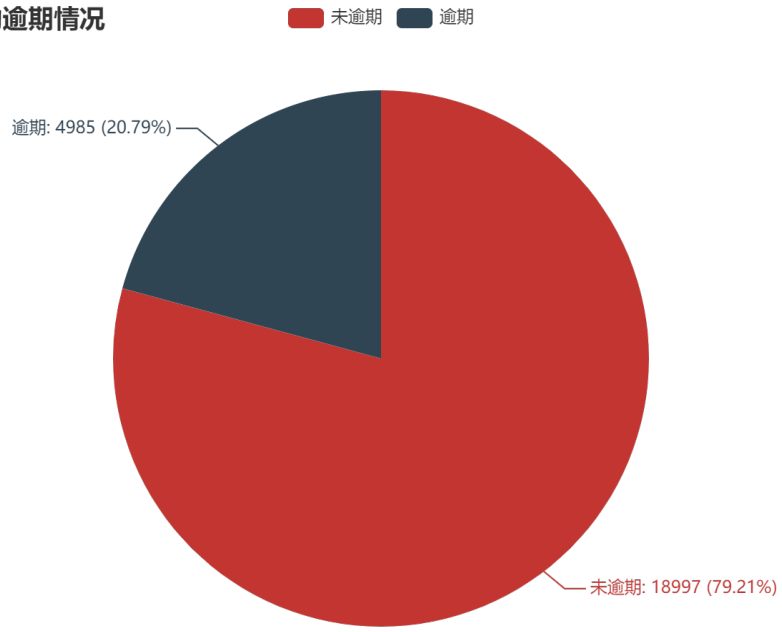


从年龄方面分析，可以得到越年轻的人，逾期的可能性越大。其中0-30年龄段逾期占比11.56%；30-45年龄段逾期占比9.38%；45-60年龄段逾期占比6.9%；60-75年龄段逾期占比3.39%；75-100年龄段占比2.01%。

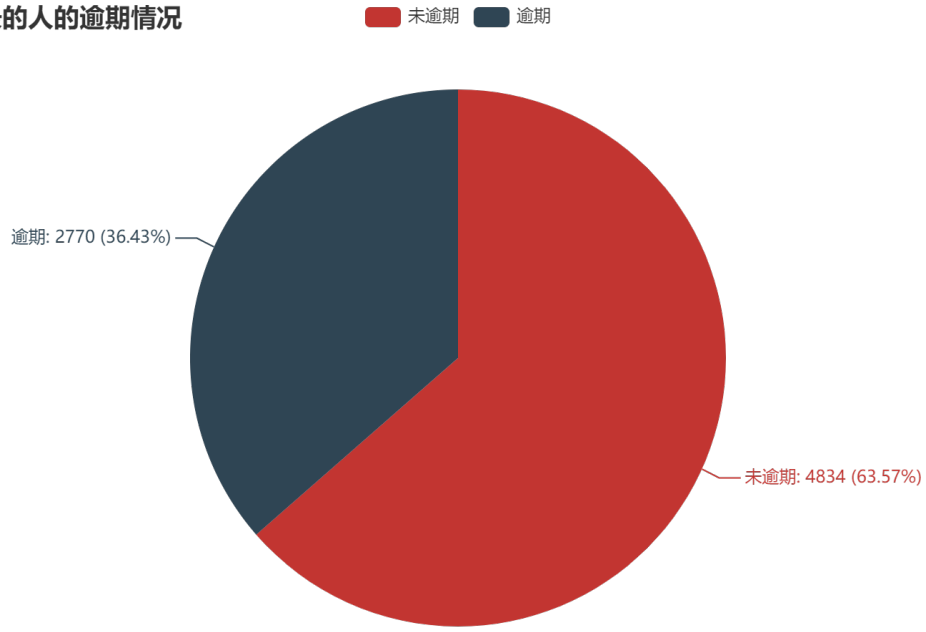
2.3 逾期记录

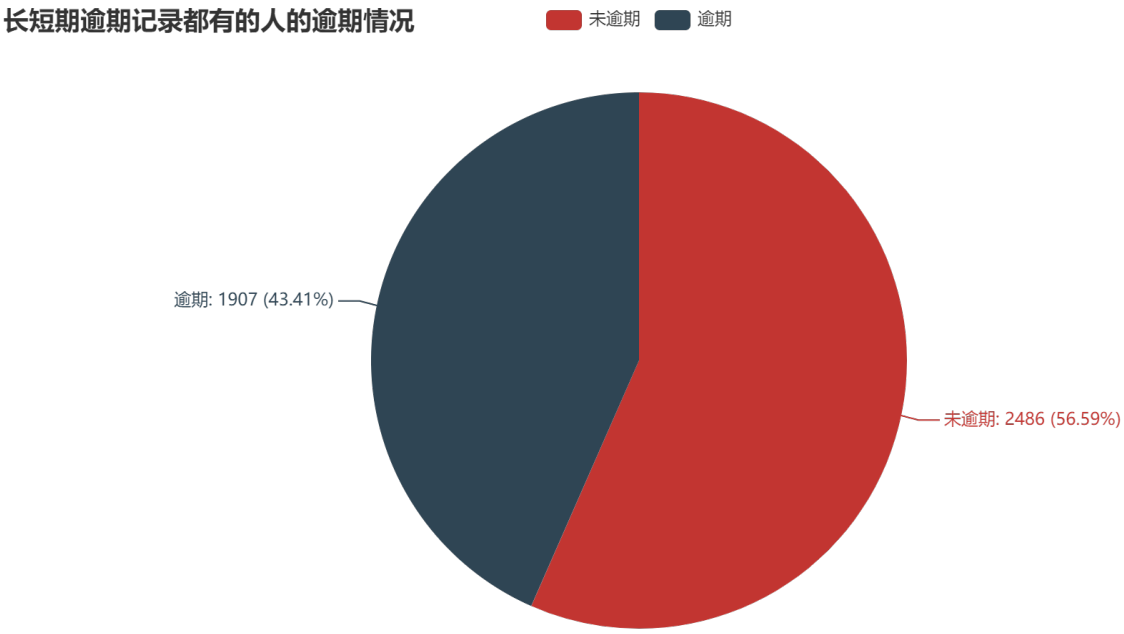


有短期逾期记录的人的逾期情况



有长期逾期记录的人的逾期情况

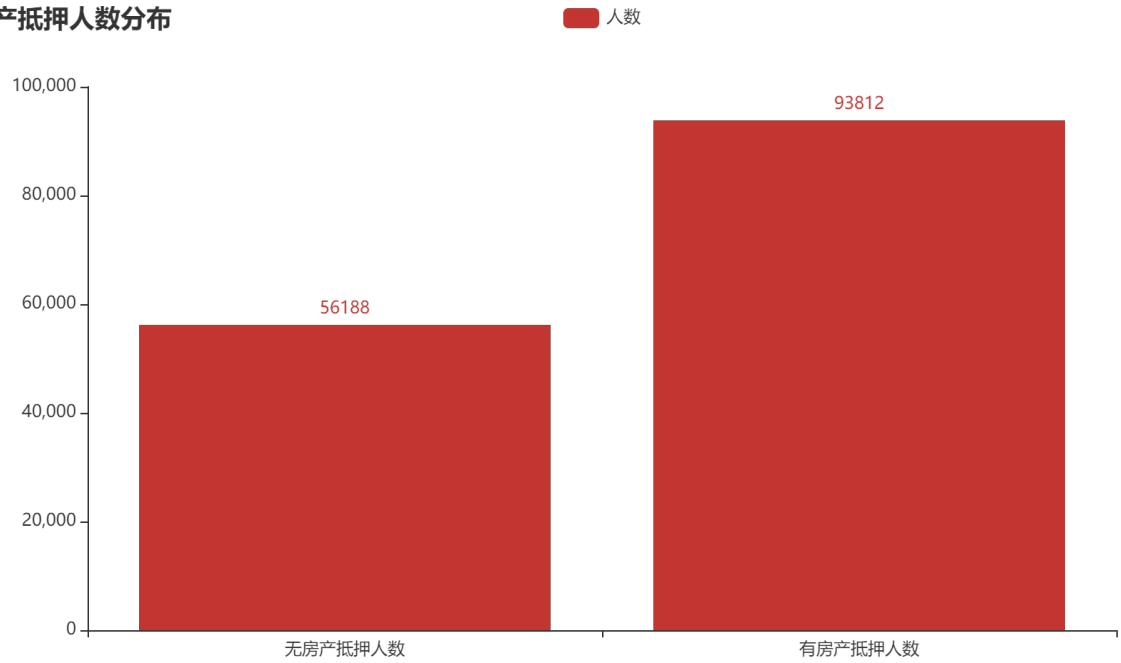




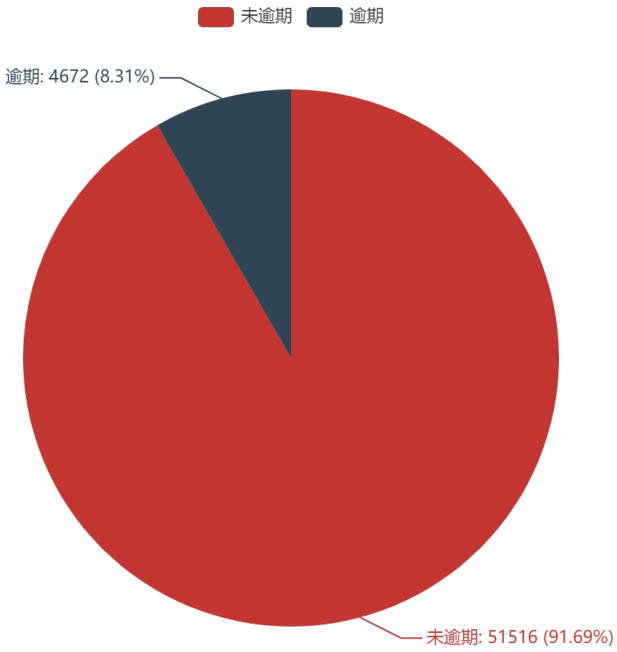
通过上方四个图，我们可以清楚的发现有逾期记录的人在本次信用卡逾期的占比大幅度上升。其中，有短期逾期记录的人，本次信用卡逾期占比20.79%；有长期逾期记录的人，本次信用卡逾期占比36.43%；有长短期逾期记录的人，本次信用卡逾期占比43.41%。针对此现象，我们可以得出，如果一个人具有逾期记录，那他信用卡逾期的可能性很大，需要慎重考虑借款给他。

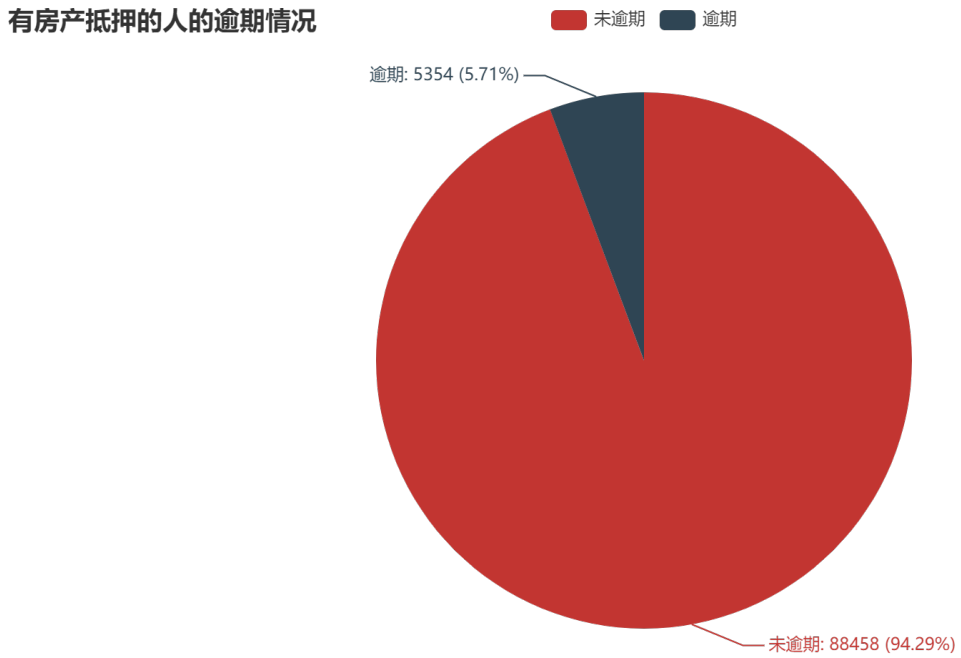
2.4 房产抵押数量

房产抵押人数分布



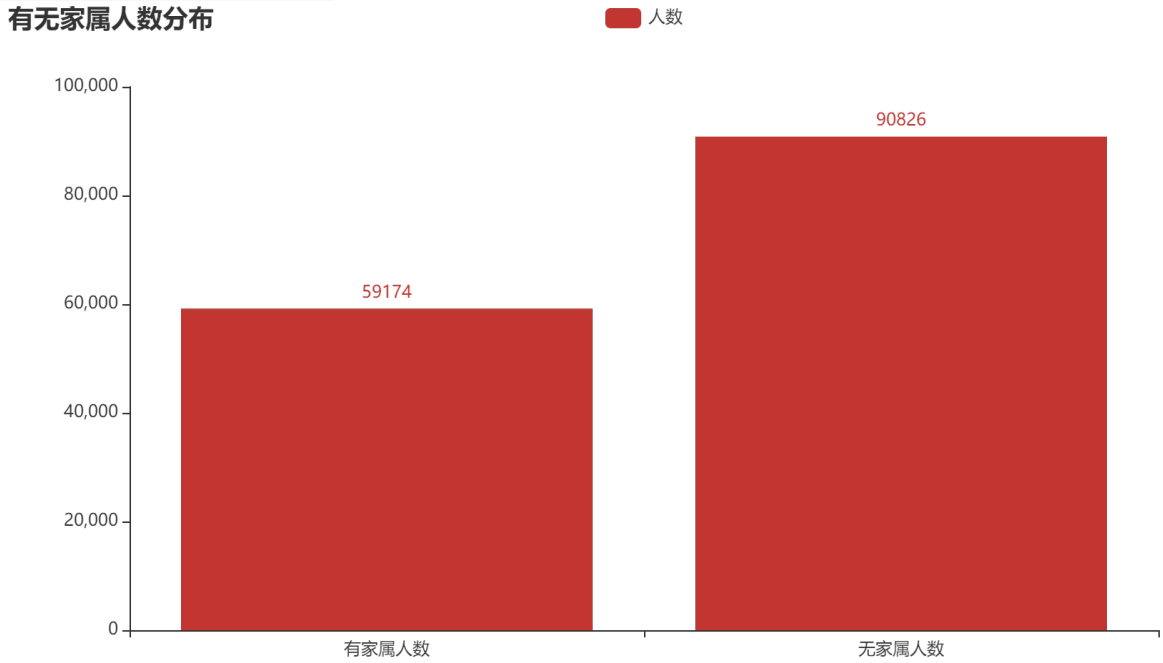
无房产抵押的人的逾期情况

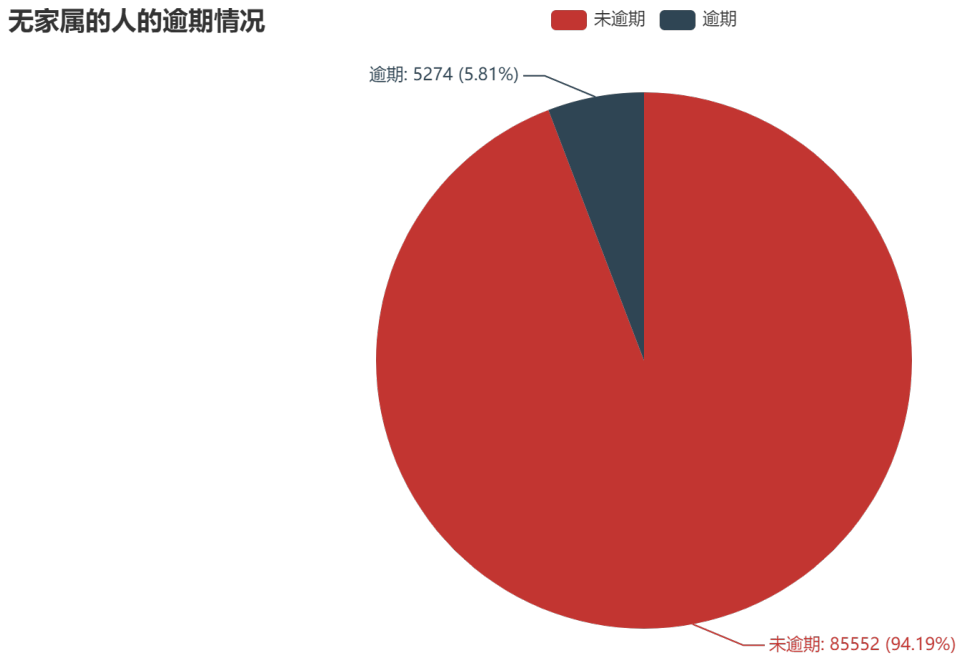


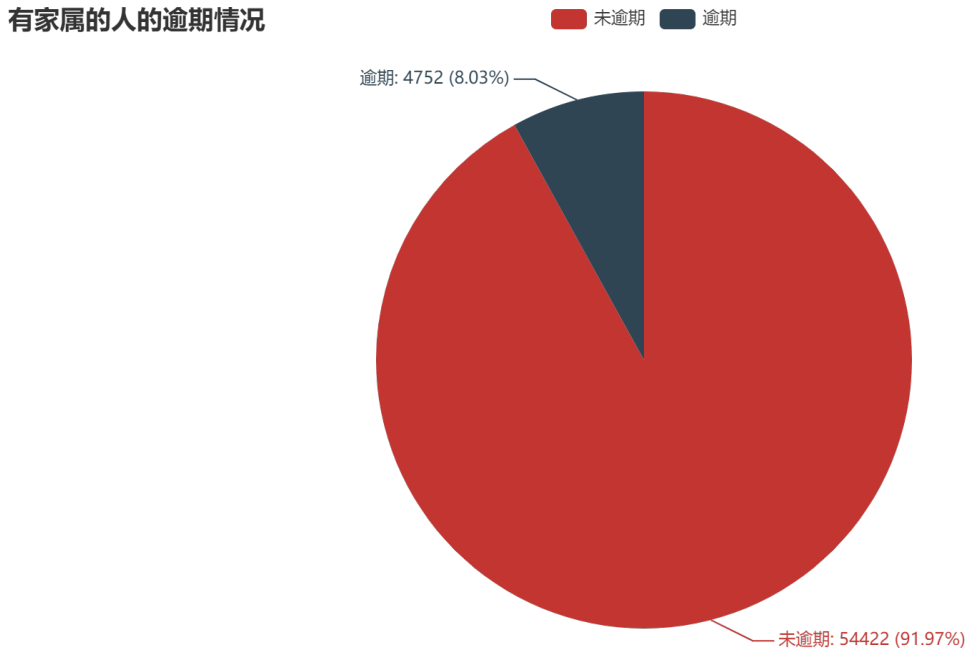


通过上方三个图，可以得到有房产抵押的人本次信用卡逾期占比5.71%，低于总体逾期占比，无房产抵押的人本次信用卡逾期占比8.31%，高于总体逾期占比。可以得出有房产抵押的人，逾期可能性稍微低一点。但占比差不多，也可以说明房产抵押这个属性对于信用卡逾期影响不大。

2.5 家属人数



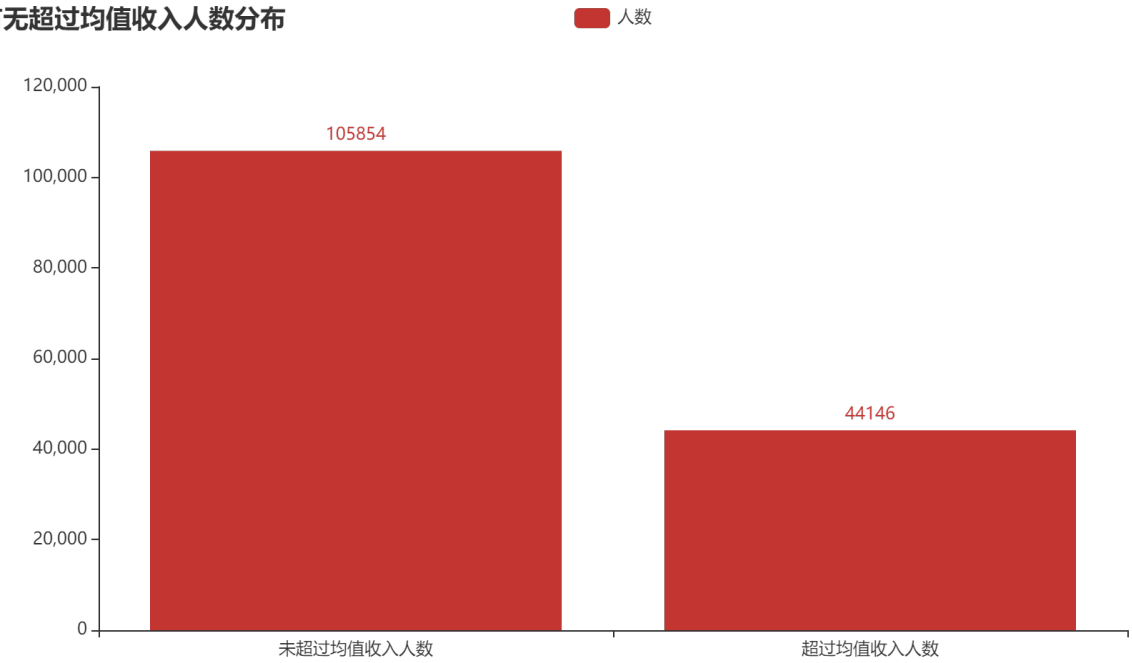




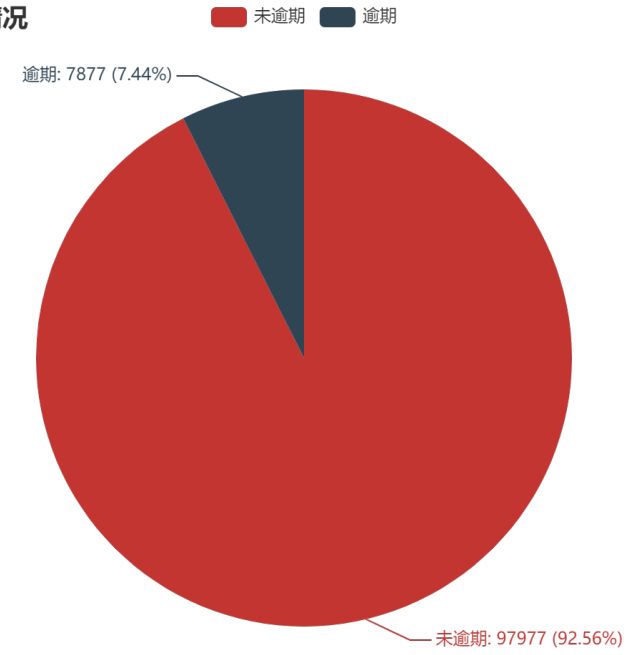
通过上方三个图，我们可以得出在家属人数属性中，有家属的人逾期可能性较大。其中无家属的人逾期占比5.81%；有家属的人逾期占比8.03%。

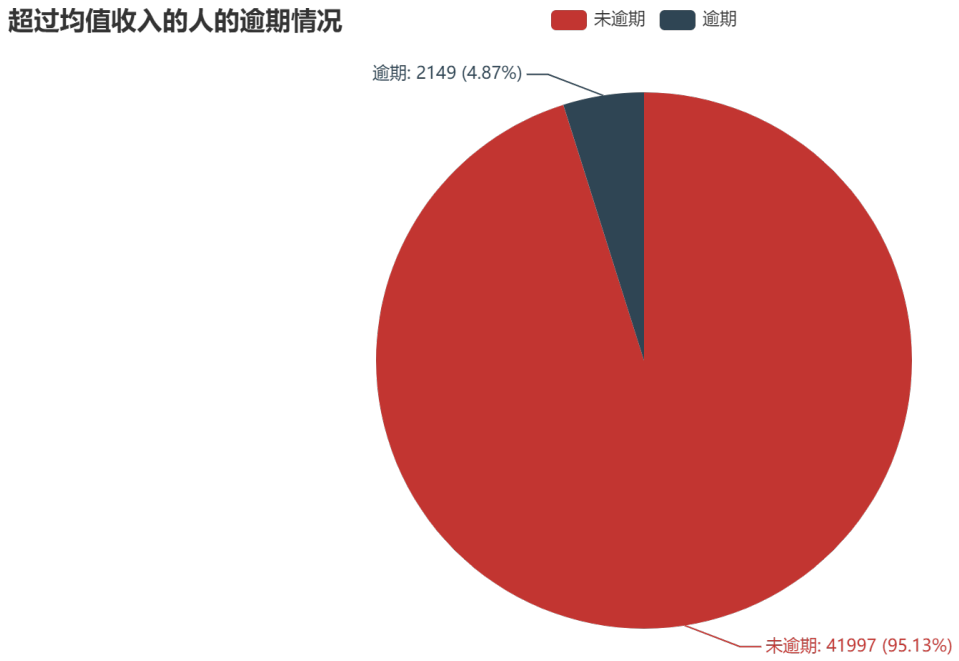
2.6 月收入

有无超过均值收入人数分布



未超过均值收入的人的逾期情况





通过上方三个图，我们可以得到月收入超过人均收入的人逾期的可能性小，其偿还贷款的能力强，月收入低于人均收入的人逾期可能性大。其中，未超过人均收入的人逾期占比7.44%；超过人均收入的人逾期占比4.87%。

3. 分析结果总结

我们从5个方面分析逾期的相关因素。通过图表，可以得到年龄和逾期记录对于本次信用卡是否逾期的影响很大。年龄越小和具有逾期记录的人，信用卡逾期的可能性越大。而是否有房产抵押、是否有家属、是否月收入超过人均水平，具有较小的相关性，影响不是很大。这些分析结果，可以在后期对其训练预测模型时，提供有效的帮助，如提高年龄和逾期记录的权重，降低其他属性的权重。